

Coding & Solution

Date	15 March 2026
Team ID	XXXXXX
Project Name	AI Learning Support Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

- Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
- The solution should address the core problem statement and implement the key functional requirements.
- Include all source files, configuration files, and setup instructions needed to run the application.
- Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	Paste your GitHub Repository URL here
Programming Language(s)	Python 3.11
Framework(s) Used	Streamlit, TensorFlow, Hugging Face Transformers, Plotly
Key Features Implemented	• Subject selection • Learning problem input • BiLSTM emotion prediction • BERT emotion prediction • Mixed emotion detection • Personalized learning guidance using Google Gemini API • Emotion confidence analysis • BiLSTM probability chart • Previous interaction history • View previous interaction details • Download interaction history (CSV) • Analytics dashboard (Emotion Distribution, Confidence Trend, Subject-wise

	Analytics) • Responsive Streamlit user interface
Pending / Incomplete Features	No major functional features pending. Minor future improvements may include enhanced analytics, database integration, and user authentication.
Setup / Run Instructions	1. Clone the repository. 2. Create and activate a Python virtual environment. 3. Install dependencies using <code>pip install -r requirements.txt</code>. 4. Configure the Google Gemini API key in the environment variables or configuration file. 5. Run the application using <code>streamlit run app.py</code>. 6. Open the displayed local URL in a web browser to use the application.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes (after the fixes and testing completed in Epic 6)
6	Code is committed to a version control repository	Yes (if you have pushed it to GitHub; otherwise mark this as No until you do)

Additional Notes / Comments:

- The application follows a modular architecture with separate service modules for **BiLSTM prediction**, **BERT prediction**, **Mixed Emotion Detection**, **Gemini API integration**, **Prediction Schema**, **Interaction History**, and **Analytics**.
- Emotion predictions are generated using both **BiLSTM** and **BERT** models, with mixed emotion detection providing primary and secondary emotion analysis.
- Personalized learning guidance is generated using the **Google Gemini API** based on the detected emotion and the student's learning problem.
- User interactions are stored in `interaction_history.csv` and can be viewed later through the **Previous Interaction History** section.

- The application includes an **Analytics Dashboard** with interactive **Plotly** visualizations, including emotion distribution, confidence trends, and subject-wise analytics.
- The project was tested for complete user flow, interaction history, analytics, responsiveness, and performance as part of **Epic 6**.

Technologies Used

- **Frontend:** Streamlit, Plotly
- **Backend:** Python 3.11
- **Deep Learning:** TensorFlow (BiLSTM)
- **Transformer Model:** Hugging Face Transformers (BERT)
- **Generative AI:** Google Gemini API
- **Libraries:** Pandas, NumPy, Scikit-learn
- **Storage:** CSV (`interaction_history.csv`), Streamlit Session State
- **Version Control:** Git, GitHub (if applicable)